

Digital Nurture 5.0

.NET Full Stack Engineer Track

HANDS-ON EXERCISE BOOK

Angular (v20.0): Components, Routing, Forms, HTTP, State Management & Testing
10 Hands-On Exercises

How to Use This Exercise Book

This exercise book covers Angular (v20.0) for the Digital Nurture 5.0 Deep Skilling Program. The 10 hands-on exercises progress from project setup and core Angular concepts through to forms, HTTP integration, state management with NgRx, and unit testing.

All exercises build a single project — a Student Course Portal — so each hands-on adds a new capability to the same application rather than starting fresh each time.

Software & Tools Required

- Node.js (LTS 20+) + npm — <https://nodejs.org/en/download/>
- Angular CLI v20 — npm install -g @angular/cli
- VS Code — <https://code.visualstudio.com/download>
- Angular Language Service (VS Code extension) — for template IntelliSense
- Chrome + Angular DevTools extension — <https://angular.dev/tools/devtools>
- JSON Server (mock REST API) — npm install -g json-server

Submission Guidelines

- Organise solutions in a folder named Angular_HandsOn/<YourName>/
- Submit the complete Angular project folder (exclude node_modules — add it to .gitignore)
- Each hands-on adds features to the same project — do not create separate projects per exercise
- Push to your GitHub repository and share the URL with your POC

NOTE	Hands-On 1 sets up the base project. Every subsequent exercise builds on top of it. Do not skip exercises — each one introduces concepts used in the next.
-------------	--

Difficulty Guide

Level	Hands-On	What to Expect
Beginner	1, 2, 3	Project setup, components, bindings, directives, pipes
Intermediate	4, 5, 6, 7	Forms, validation, services, DI, routing, guards
Advanced	8, 9, 10	HTTP client, interceptors, NgRx, RxJS, unit testing

Common Scenario: Student Course Portal

All exercises build one unified Angular application — a Student Course Portal for a college system. Each hands-on adds a new layer to the portal, so by the end you will have a complete, production-style Angular SPA with routing, forms, HTTP integration, state management, and test coverage.

The Student Course Portal allows students to view their profile, browse and enroll in courses, check grades, and receive notifications. Administrators can manage course listings. You will build this portal incrementally — starting from a scaffolded project and progressively adding Angular features with each hands-on.

Portal Pages & Features

Page / Feature	Built In	Key Angular Concepts
Home / Dashboard	Hands-On 1 & 2	Components, property binding, lifecycle hooks
Course Listing	Hands-On 3	ngFor, ngIf, custom pipes, attribute directives
Enrollment Form	Hands-On 4 & 5	Template-driven & reactive forms, validation
Navigation	Hands-On 6 & 7	Services, DI, routing, route guards
API Integration	Hands-On 8	HttpClient, interceptors, error handling
State Management	Hands-On 9	NgRx store, actions, reducers, effects, selectors
Testing	Hands-On 10	Jasmine, Karma, component & service tests

HANDS-ON 1 [Beginner]**Environment Setup, Project Structure & First Component****Topics Covered:**

✓ Angular CLI & Workspace Setup	✓ Angular Project Structure & Files
✓ Running & Building Angular Apps	✓ Creating Components
✓ Angular Module Overview	

In this first hands-on, you will install the Angular CLI, scaffold the Student Course Portal project, understand every generated file and folder, and create your first Angular component from scratch.

SET UP	<code>npm install -g @angular/cli then ng new student-course-portal --routing --style=css then cd student-course-portal && ng serve</code>
---------------	--

Task 1: Scaffold and Explore the Angular Project

Goal: Create the Angular workspace and understand what each generated file does.

Steps:

1. Run `ng new student-course-portal --routing --style=css`. Accept defaults for all prompts.
2. Open the project in VS Code. For each of the following files, write a one-line comment in a `notes.txt` file explaining its purpose: `angular.json`, `tsconfig.json`, `tsconfig.app.json`, `package.json`, `src/main.ts`, `src/app/app.module.ts` (or `app.config.ts` in standalone mode), `src/app/app.component.ts`, `src/index.html`.
3. Run `ng serve` and open `http://localhost:4200`. Verify the default Angular welcome page loads.
4. Run `ng build` and inspect the `dist/` folder. Note the generated files — identify which file contains the compiled application code.
5. In `angular.json`, locate the budgets configuration under `architect > build > configurations > production`. Note the `maximumWarning` and `maximumError` thresholds — document what these control.

Hint:

- Angular 17+ defaults to standalone components (no `NgModules` required). Angular 20 continues this pattern. If you see `app.config.ts` instead of `app.module.ts`, your project is using the standalone API — which is the modern approach.
- `src/main.ts` is the application entry point — it bootstraps either the `AppModule` or the root component directly.

Expected Outcome: `ng serve` runs without errors. `notes.txt` contains explanations for all 8 files. `ng build` produces a `dist/` folder.

Task 2: Create and Organise Components

Goal: Generate the core page components for the Student Course Portal.

Steps:

6. Generate four components using Angular CLI: `ng generate component components/header`, `ng generate component pages/home`, `ng generate component pages/course-list`, `ng generate component pages/student-profile`. Observe the 4 files generated for each: `.ts`, `.html`, `.css`, `.spec.ts`.
7. In `HeaderComponent`'s template (`header.component.html`), add a `<nav>` with the portal name 'Student Course Portal' and placeholder links: Home, Courses, Profile.
8. In `HomeComponent`'s template, add a welcome heading (`<h1>`), a short description paragraph, and a stats row showing three hardcoded values: Courses Available: 12, Enrolled: 3, GPA: 3.8.
9. Add `<app-header>` and `<router-outlet>` to `app.component.html`, replacing the default boilerplate content.
10. Verify `ng serve` still compiles without errors and the header and home page content are visible.

Hint:

- ng generate component automatically adds the component to the nearest module's declarations (or registers it as standalone in modern Angular). You do not need to register it manually.
- Each component generated by the CLI follows the same structure: @Component decorator with selector, templateUrl, and styleUrls — understand each property before moving on.

Expected Outcome: Browser shows the header nav and home page welcome content. All 4 components generated without errors.

HANDS-ON 2 [Beginner]**Data Binding, Lifecycle Hooks & Component Communication****Topics Covered:**

✓ Property Binding	✓ Event Binding
✓ Two-Way Binding (ngModel)	✓ Lifecycle Hooks — ngOnInit, ngOnChanges, ngOnDestroy
✓ @Input and @Output Decorators	✓ EventEmitter

You will now make the Student Course Portal dynamic using Angular's four binding types, implement lifecycle hooks to control component behaviour, and establish parent-child communication using @Input and @Output.

Task 1: All Four Binding Types

Goal: Practise interpolation, property binding, event binding, and two-way binding in one component.

Steps:

11. In HomeComponent, declare a TypeScript property: portalName = 'Student Course Portal'. Use string interpolation in the template to display it: `<h1>{{ portalName }}</h1>`.
12. Add a property isPortalActive = true. Use property binding to control a button's disabled state: `<button [disabled]='!isPortalActive'>Enroll Now</button>`.
13. Add a method onEnrollClick() that sets a property message = 'Enrollment opened!'. Use event binding on the button: `(click)='onEnrollClick()'`.
14. Add an `<input>` for a search term and a property searchTerm = ''. Use two-way binding with ngModel: `<input [(ngModel)]='searchTerm'>`. Import FormsModule in the module/component. Display the live value below the input: `<p>Searching for: {{ searchTerm }}</p>`.
15. Explain in a code comment the difference between [property] (one-way, component → DOM) and [(ngModel)] (two-way, DOM ↔ component).

Hint:

- Two-way binding [(ngModel)] is shorthand for [ngModel]='prop' (ngModelChange)='prop=\$event' — you can always split it into one-way bindings if you need to intercept the change.
- FormsModule must be imported in the component's imports array (standalone) or the module's imports (NgModule) for ngModel to work.

Expected Outcome: Typing in the search box updates the 'Searching for:' text in real time. Clicking Enroll Now shows the message. Button is disabled/enabled based on isPortalActive.

Task 2: Lifecycle Hooks

Goal: Implement the three most important lifecycle hooks and observe when they fire.

Steps:

16. In HomeComponent, implement ngOnInit: fetch (or simulate) a count of available courses and log 'HomeComponent initialised — courses loaded' to the console.
17. Implement ngOnDestroy: log 'HomeComponent destroyed' to the console. Navigate away from the home page and back — observe both logs in the browser console.
18. Create a new CourseCardComponent: ng generate component components/course-card. Add an @Input() course: any property. Implement ngOnChanges(changes: SimpleChanges): log the previous and current value of the course input whenever it changes.
19. In CourseListComponent, render three CourseCardComponents with different hardcoded course inputs. Open the console and verify ngOnChanges fires once per card on initial render.

Hint:

- `ngOnInit` fires once after the component's inputs are first set — use it for data fetching, not the constructor (which fires before inputs are set).
- `ngOnDestroy` is critical for unsubscribing from Observables and clearing timers — forgetting it causes memory leaks in long-running SPAs.

Expected Outcome: Console shows `ngOnInit` log on Home page load, `ngOnDestroy` log on navigation away, and `ngOnChanges` logs with previous/current values for each card.

| Task 3: @Input and @Output — Parent-Child Communication

Goal: Pass data down with `@Input` and emit events up with `@Output`.

Steps:

20. In `CourseCardComponent`, add `@Input() course: { id: number, name: string, code: string, credits: number }` and render all four fields in the template.
21. Add an `@Output() enrollRequested = new EventEmitter<number>()` to `CourseCardComponent`. Add an 'Enroll' button in the template: `<button (click)=enrollRequested.emit(course.id)>Enroll</button>`.
22. In `CourseListComponent`, define a `courses` array with 5 course objects. Render a `CourseCardComponent` for each: `<app-course-card *ngFor='let c of courses' [course]='c' (enrollRequested)='onEnroll($event)'></app-course-card>`.
23. Implement `onEnroll(courseId: number)` in `CourseListComponent`: log 'Enrolling in course: ' + `courseId` and update a `selectedCourseId` property.
24. Display the `selectedCourseId` below the list: `<p *ngIf='selectedCourseId'>Selected course ID: {{ selectedCourseId }}</p>`.

Hint:

- `@Input` and `@Output` create a one-way data flow: data flows down via `@Input`, events bubble up via `@Output`. This is Angular's recommended pattern for parent-child communication.
- `@Output` with `EventEmitter<T>` where `T` is the payload type — using the generic type makes the event strongly typed.

Expected Outcome: Clicking Enroll on any card logs the course ID to the console and shows the selected ID below the list.

HANDS-ON 3 [Beginner]**Directives & Pipes — Built-in and Custom****Topics Covered:**

✓ Structural Directives — *ngIf, *ngFor, *ngSwitch	✓ Attribute Directives — ngClass, ngStyle
✓ Custom Attribute Directive	✓ Built-in Pipes — date, uppercase, currency, number
✓ Custom Pipe with PipeTransform	

Directives extend HTML with Angular-specific behaviour and pipes transform displayed values. In this hands-on, you will use all built-in structural and attribute directives, and build one custom directive and one custom pipe for the Student Course Portal.

Task 1: Structural Directives

Goal: Control DOM structure dynamically using *ngIf, *ngFor, and *ngSwitch.

Steps:

- In CourseListComponent, add a boolean property isLoading = true. Use *ngIf to show a <p>Loading courses...</p> when isLoading is true, and show the course grid when it is false. After 1.5 seconds (use setTimeout in ngOnInit), set isLoading = false.
- Use *ngFor to render the CourseCardComponents: <app-course-card *ngFor='let course of courses; let i = index; trackBy: trackByCourseId'>. Implement the trackByCourseId(index, course) method returning course.id — explain in a comment why trackBy improves performance.
- Add a gradeStatus property to each course object: 'passed', 'failed', or 'pending'. Use *ngSwitch in CourseCardComponent to display a different badge for each status: green 'Passed', red 'Failed', grey 'Pending'.
- Add a *ngIf with an else block: <ng-template #noCourses><p>No courses available.</p></ng-template>. Show this template when the courses array is empty.

Hint:

- *ngIf with else uses Angular's ng-template — the else block is never added to the DOM when the condition is true, unlike CSS display:none which hides but keeps the element.
- trackBy is essential for large lists — without it, Angular re-renders every list item on any array change. With trackBy, only changed items are updated.

Expected Outcome: Loading message shows for 1.5 seconds then disappears. trackBy is implemented. Switch renders the correct badge per grade status. Empty array shows the no-courses template.

Task 2: Attribute Directives — ngClass and ngStyle

Goal: Apply dynamic CSS classes and inline styles using attribute directives.

Steps:

- In CourseCardComponent, use [ngClass] to apply a CSS class conditionally: apply a 'card--enrolled' class when the course has been enrolled, and 'card--full' when credits >= 4. Define both classes in course-card.component.css.
- Use [ngStyle] to set the card's left border colour dynamically based on gradeStatus: green for 'passed', red for 'failed', grey for 'pending'.
- Add an isExpanded property to CourseCardComponent. Use [ngClass]='{"expanded": isExpanded}' to toggle a class that increases the card height. Toggle isExpanded on a 'Show Details' button click.
- Refactor the ngClass object binding to use a getter in the component class: get cardClasses() { return { ... } } — bind it as [ngClass]='cardClasses'. Explain in a comment why getters keep templates clean.

Hint:

- [ngClass] accepts a string, array, or object. The object form { 'className': condition } is the most readable for conditional classes.
- Prefer [ngClass] over [ngStyle] for styling — classes keep styles in CSS files where they belong. Use [ngStyle] only for truly dynamic values that cannot be expressed as a class.

Expected Outcome: Enrolled cards have a visible 'card--enrolled' style. Grade status is reflected by border colour. Show Details toggles the expanded class.

| Task 3: Custom Directive and Custom Pipe

Goal: Build a reusable attribute directive and a custom pipe for the portal.

Steps:

33. Generate a custom directive: ng generate directive directives/highlight. The directive should add a yellow background to any element it is applied to on mouseenter, and remove it on mouseleave. Use @HostListener for both events.
34. Apply the directive to course cards in the template: <app-course-card appHighlight ...>. Verify the hover highlight works.
35. Generate a custom pipe: ng generate pipe pipes/credit-label. The pipe should transform a credits number into a human-readable string: 1 → '1 Credit', 2+ → '2 Credits'. Handle edge cases (null, 0) by returning 'No Credits'.
36. Use the pipe in CourseCardComponent's template: {{ course.credits | creditLabel }}. Test with credits = 1, credits = 3, and credits = null.
37. Make the highlight directive configurable: add @Input() appHighlight = 'yellow' so the caller can pass a custom colour: <app-course-card appHighlight='lightblue'>.

Hint:

- @HostListener('mouseenter') binds to host element events without needing to manually add/remove event listeners — Angular handles cleanup automatically.
- Pipes are pure by default (only re-run when the input reference changes). If your pipe needs to re-run on mutable data changes, set pure: false in the @Pipe decorator — but use this sparingly as it impacts performance.

Expected Outcome: Hovering over a course card shows a yellow highlight. {{ 3 | creditLabel }} displays '3 Credits'. The configurable colour works via input binding.

HANDS-ON 4 [Intermediate]**Template-Driven Forms & Validation****Topics Covered:**

✓ Template-Driven Form Basics	✓ ngModel Two-Way Binding in Forms
✓ Built-in Validators — required, minlength, email	✓ ngForm and ngModel References
✓ Displaying Validation Errors	✓ Form Submission Handling

Template-driven forms are Angular's simpler, HTML-centric form approach. You will build a Student Enrollment Request form using template-driven forms, add built-in validation, display contextual error messages, and handle form submission.

IMP ORT	Add FormsModule to the component's imports array (standalone) or the module's imports to enable ngModel and ngForm.
--------------------	---

Task 1: Build the Enrollment Request Form

Goal: Create a template-driven form with multiple input types and ngModel bindings.

Steps:

38. Generate an EnrollmentFormComponent: `ng generate component pages/enrollment-form`. Add it as a route in `app-routing.module.ts` at path `/enroll`.
39. In the template, build a form using `<form #enrollForm='ngForm' (ngSubmit)='onSubmit(enrollForm)'>`. Add the following fields, each with `[(ngModel)]` and a name attribute: `studentName` (text input), `studentEmail` (email input), `courseId` (number input), `preferredSemester` (select with options: Odd, Even), `agreeToTerms` (checkbox).
40. In the component class, define an `onSubmit(form: NgForm)` method that logs `form.value` and `form.valid` to the console.
41. Add a Submit button: `<button type='submit' [disabled]='enrollForm.invalid'>Submit</button>`.
42. Test the form: submit with all fields filled and observe the logged values. Note the structure of `form.value`.

Hint:

- The `#enrollForm='ngForm'` template reference variable gives you access to the `NgForm` directive instance — use it to check form validity, reset the form, or access individual control states.
- `name` attribute is mandatory on every form control when using template-driven forms — Angular uses it as the key in `form.value`.

Expected Outcome: Console logs the form value object with all field names as keys on submit. Submit button is disabled when the form is invalid.

Task 2: Validation and Error Messages

Goal: Add built-in validators and display contextual error messages for each field.

Steps:

43. Add validation attributes to `studentName`: `required` and `minlength='3'`. Add `required` and `email` to `studentEmail`. Add `required` to `courseId`. Add `required (Checked)` to `agreeToTerms`.
44. For `studentName`, display error messages conditionally: show 'Name is required' when the control is touched and has the required error; show 'Name must be at least 3 characters' when it has the `minlength` error. Use the pattern: `<span *ngIf='nameCtrl.touched && nameCtrl.errors?.required'>.</span>` after adding `#nameCtrl='ngModel'` to the input.
45. Apply Angular's form state CSS classes to style inputs: add CSS rules for `.ng-invalid.ng-touched` { `border-color: red;` } and `.ng-valid.ng-touched` { `border-color: green;` } in the component's stylesheet.

46. Add a success message `<div>` that appears after successful submission: 'Enrollment request submitted successfully!' Use a submitted boolean property toggled in `onSubmit`, and `*ngIf` to show/hide it.
47. Add a Reset button that calls `enrollForm.resetForm()` to clear all fields and validation states.

Hint:

- `touched` is true once the user has focused and left the field. `dirty` is true once the user has changed the value. Use `touched` for showing errors — not `dirty` — so errors show after the user interacts with a field, not while typing.
- Angular automatically adds CSS classes to form controls: `ng-valid/ng-invalid` (validation state), `ng-pristine/ng-dirty` (changed state), `ng-touched/ng-untouched` (visited state).

Expected Outcome: Invalid fields show red borders and specific error messages after being touched. Valid fields show green borders. Successful submission shows the success message.

HANDS-ON 5 [Intermediate]**Reactive Forms — FormBuilder, FormGroup, FormArray & Custom Validators****Topics Covered:**

✓ ReactiveFormsModule Setup	✓ FormBuilder & FormGroup
✓ FormControl & FormArray	✓ Built-in Validators
✓ Custom Synchronous Validators	✓ Custom Async Validators
✓ Dynamic Form Controls	

Reactive forms define form structure in the component class rather than the template — making them more powerful, testable, and suited to complex scenarios. You will rebuild the enrollment form as a reactive form and add custom validators and dynamic controls.

IMP ORT	Add ReactiveFormsModule to the component's imports array (standalone) or the module's imports to enable FormBuilder, FormGroup, and FormControl.
--------------------	--

Task 1: Build a Reactive Form with FormBuilder

Goal: Define and bind a reactive form using FormBuilder.

Steps:

48. Generate a ReactiveEnrollmentFormComponent: `ng generate component pages/reactive-enrollment-form`. Add a route at `/enroll-reactive`.
49. Inject FormBuilder in the constructor: `constructor(private fb: FormBuilder) {}`. In `ngOnInit`, build the form: `this.enrollForm = this.fb.group({ studentName: ['', [Validators.required, Validators.minLength(3)]], studentEmail: ['', [Validators.required, Validators.email]], courseId: [null, Validators.required], preferredSemester: ['Odd', Validators.required], agreeToTerms: [false, Validators.requiredTrue] });`
50. Bind the form in the template using `[formGroup]='enrollForm'`. Bind each input using `formControlName='studentName'` (etc.). No `ngModel` is needed in reactive forms.
51. Add a Submit button: `<button [disabled]='enrollForm.invalid'>Submit</button>`. On submit, log `enrollForm.value` and `enrollForm.getRawValue()`.
52. Explain in a comment the difference between `enrollForm.value` (excludes disabled controls) and `enrollForm.getRawValue()` (includes all controls).

Hint:

- In reactive forms, the form model lives entirely in TypeScript — the template just binds to it. This makes the form logic fully unit-testable without a DOM.
- `Validators.requiredTrue` specifically validates that a checkbox is checked — `Validators.required` only checks for a non-empty/non-null value.

Expected Outcome: Form renders correctly bound to the reactive model. Submit logs the form value. Disabled Submit button enables only when all validators pass.

Task 2: Custom Validators and FormArray for Dynamic Controls

Goal: Write a custom synchronous validator and use FormArray for repeating controls.

Steps:

53. Write a custom validator function `noCourseCode(control: AbstractControl): ValidationErrors | null` that returns `{ noCourseCode: true }` if the control value starts with 'XX' (a disallowed prefix). Apply it to the `courseId` control alongside `Validators.required`.
54. Display the custom error: `<span *ngIf='enrollForm.get("courseId")?.errors?.["noCourseCode"]'>Course code starting with XX is not allowed.</span>`.

55. Write an async validator `simulateEmailCheck` that returns a Promise: after 800ms, if the email contains 'test@', return `{ emailTaken: true }`, otherwise return null. Apply it as the third argument of the email FormControl: `this.fb.control("", [Validators.required, Validators.email], [simulateEmailCheck])`.
56. Add a `FormArray` for `additionalCourses`: `this.fb.array([])`. Add an 'Add Another Course' button that pushes a new `FormControl("", Validators.required)` into the array. Render the array: `<div *ngFor='let ctrl of additionalCourses.controls; let i=index'><input [formControl]='ctrl'><button (click)='removeCourse(i)'>Remove</button></div>`.
57. Implement `get additionalCourses()` `{ return this.enrollForm.get('additionalCourses') as FormArray; }` as a typed getter — explain why this getter is better than casting in the template.

Hint:

- Async validators fire after all sync validators pass (to avoid unnecessary API calls). They return `Observable<ValidationErrors | null>` or `Promise<ValidationErrors | null>`.
- `FormArray` is Angular's answer to dynamic, repeating form sections — like adding multiple phone numbers or, in this case, enrolling in multiple courses at once.

Expected Outcome: Entering 'XX101' in `courseId` shows the custom error. `test@example.com` shows 'Email taken' after 800ms. Additional course controls can be added and removed dynamically.

HANDS-ON 6 [Intermediate]**Services & Dependency Injection****Topics Covered:**

✓ Creating and Providing Services	✓ providedIn: 'root' vs Component-level Providers
✓ Injecting Services into Components	✓ Service as a Shared Data Store
✓ Hierarchical Dependency Injection	✓ Singleton Service Pattern

Services centralise shared logic and data that does not belong in a single component. In this hands-on, you will create Angular services for the Student Course Portal, understand the DI hierarchy, and use a service as a shared state store between components.

Task 1: Create and Use a Course Service

Goal: Build a CourseService that provides course data to multiple components.

Steps:

58. Generate a service: `ng generate service services/course`. Provide it at the root level: `@Injectable({ providedIn: 'root' })`. In the service, define a private `courses` array of 5 course objects (same structure as before) and the following methods: `getCourses(): Course[]`, `getCourseById(id: number): Course | undefined`, `addCourse(course: Course): void`.
59. Define a `Course` interface in a separate file (`models/course.model.ts`): `export interface Course { id: number; name: string; code: string; credits: number; gradeStatus: 'passed' | 'failed' | 'pending'; }`.
60. Inject `CourseService` into `CourseListComponent` via the constructor: `constructor(private courseService: CourseService) {}`. In `ngOnInit`, replace the hardcoded `courses` array with `this.courseService.getCourses()`.
61. Inject the same `CourseService` into `HomeComponent` and use `getCourses().length` to display the live courses count in the stats row.
62. Add a second component (e.g., a `CourseSummaryWidget`) that also injects `CourseService` and calls `getCourses()`. Verify both components share the same service instance by adding a course in one and observing the count update in the other.

Hint:

- `providedIn: 'root'` makes the service a singleton — one instance shared across the entire application. All injectors up the tree share the same instance.
- Defining a TypeScript interface for your data models (`Course`, `Student`) gives you compile-time type checking across the entire application — always prefer this over using `'any'`.

Expected Outcome: Course list and home stats both read from the same service. Adding a course in one component is reflected in the other's count — confirming the singleton instance.

Task 2: Enrollment Service and Hierarchical DI

Goal: Create an `EnrollmentService` and demonstrate the DI hierarchy with component-level providers.

Steps:

63. Generate an `EnrollmentService`: `ng generate service services/enrollment`. Provide it at root level. Add: `private enrolledCourseIds: number[] = []`; with methods `enroll(courseId: number): void`, `unenroll(courseId: number): void`, `isEnrolled(courseId: number): boolean`, `getEnrolledCourses(): Course[]` (use `CourseService` internally to resolve IDs to full `Course` objects).
64. Inject `CourseService` into `EnrollmentService` — this demonstrates service-to-service injection.
65. Inject `EnrollmentService` into `CourseCardComponent`. When the `Enroll` button is clicked, call `enrollmentService.enroll(course.id)`. Use `isEnrolled(course.id)` to toggle the button label between `'Enroll'` and `'Unenroll'`.

66. In `StudentProfileComponent`, inject `EnrollmentService` and display the list of enrolled courses using `getEnrolledCourses()`.
67. Demonstrate component-level providing: in a new `NotificationComponent`, provide a `NotificationService` at component level using providers: `[NotificationService]` in the `@Component` decorator. Explain in a comment why this creates a new, separate instance scoped to that component.

Hint:

- Component-level providers create a new service instance for that component and its children — useful when you need isolated state per component instance, such as a form wizard with multiple steps.
- Injecting one service into another is a powerful pattern — services can depend on other services, creating a layered architecture similar to a backend service layer.

Expected Outcome: Enroll toggles to Unenroll after clicking. Enrolled courses appear in the profile page. The comment explains the scoped `NotificationService` instance.

HANDS-ON 7 [Intermediate]**Angular Routing — Guards, Lazy Loading & Route Data****Topics Covered:**

✓ Configuring Routes & RouterModule	✓ Route Parameters & Query Parameters
✓ Nested Routes	✓ Lazy Loading Feature Modules
✓ CanActivate Guard	✓ CanDeactivate Guard
✓ Passing Data Between Routes with Resolve	

Routing is what makes Angular a true SPA framework. In this hands-on, you will configure all portal routes, add route parameters, implement lazy loading for the enrollment module, and protect routes with guards.

Task 1: Route Configuration, Parameters and Nested Routes

Goal: Set up all portal routes including dynamic segments and nested routes.

Steps:

68. In `app-routing.module.ts` (or `app.routes.ts` in standalone), define the following routes: `{ path: '' } → HomeComponent`, `{ path: 'courses' } → CourseListComponent`, `{ path: 'courses/:id' } → CourseDetailComponent` (generate this component), `{ path: 'profile' } → StudentProfileComponent`, `{ path: '**' } → a NotFoundComponent` (generate it).
69. Generate `CourseDetailComponent`: `ng generate component pages/course-detail`. In it, inject `ActivatedRoute` and read the `:id` parameter: `this.route.snapshot.paramMap.get('id')`. Use `CourseService.getCourseById()` to load and display the matching course.
70. In `CourseListComponent`, make each course card clickable — navigate to `/courses/:id` using `Router.navigate(['courses', course.id])` when the card is clicked.
71. Add a query parameter to the courses route: when the search input has a value, update the URL to `/courses?search=angular` using `this.router.navigate(['courses'], { queryParams: { search: this.searchTerm } })`. Read it back in `ngOnInit` using `this.route.snapshot.queryParamMap.get('search')`.
72. Add nested routes under `/courses`: `{ path: 'courses', component: CoursesLayoutComponent, children: [{ path: '', component: CourseListComponent }, { path: ':id', component: CourseDetailComponent }] }`. Add a `<router-outlet>` to `CoursesLayoutComponent`.

Hint:

- `route.snapshot.paramMap` is fine for parameters that do not change while the component is active. For parameters that can change (same component, different params), subscribe to `route.paramMap` `Observable` instead.
- The `**` wildcard route must always be the last route — Angular matches routes in order, and a wildcard before specific routes would catch everything.

Expected Outcome: Navigating to `/courses/2` shows the correct course detail. Search updates the URL query param. The 404 page appears for unknown routes.

Task 2: Lazy Loading and Route Guards

Goal: Implement lazy loading for the enrollment feature and protect routes with guards.

Steps:

73. Create a feature module for enrollment: `ng generate module features/enrollment --routing`. Move `EnrollmentFormComponent` and `ReactiveEnrollmentFormComponent` into this module. Set up lazy loading in the main app routes: `{ path: 'enroll', loadChildren: () => import('./features/enrollment/enrollment.module').then(m => m.EnrollmentModule) }`.
74. Verify lazy loading works: open Chrome DevTools → Network tab → navigate to `/enroll` — a separate JavaScript chunk file should be downloaded only when the route is first visited, not on initial app load.

75. Generate an auth guard: `ng generate guard guards/auth`. Implement `canActivate`: check if a property `isLoggedIn = true` (hardcoded for now) in an `AuthService`. Return `true` if logged in, otherwise navigate to `'/'` and return `false`.
76. Apply the guard to the `/profile` and `/enroll` routes: `{ path: 'profile', canActivate: [AuthGuard], component: StudentProfileComponent }`.
77. Generate a `CanDeactivate` guard for the reactive enrollment form: `ng generate guard guards/unsaved-changes`. Implement `canDeactivate` to check if the form is dirty (`enrollForm.dirty`). If dirty, use `window.confirm('You have unsaved changes. Leave?')` — return `true` to allow navigation, `false` to stay.

Hint:

- Lazy loading splits the application into chunks that are downloaded on demand — critical for large applications. Without it, all code is bundled into one large file downloaded on initial load.
- `CanDeactivate` guards prevent accidental loss of form data — one of the most user-friendly features you can add to any form-heavy application.

Expected Outcome: Network tab shows a separate chunk loaded on first `/enroll` navigation. Navigating to `/profile` without login redirects to home. Dirty form shows a confirmation dialog on navigation away.

HANDS-ON 8 [Advanced]**HTTP Client — API Integration, Observables & Interceptors****Topics Covered:**

✓ Setting Up HttpClientModule	✓ GET, POST, PUT, DELETE with HttpClient
✓ Observables vs Promises	✓ RxJS Operators — map, catchError, tap, switchMap
✓ HTTP Interceptors	✓ Error Handling & Retry Strategies

Real Angular applications communicate with backend APIs. In this hands-on, you will replace all hardcoded service data with live HTTP calls, handle errors gracefully, and add interceptors for cross-cutting concerns like auth tokens and logging.

SET UP	Install JSON Server to mock the backend: <code>npm install -g json-server</code> . Create <code>db.json</code> with <code>courses</code> , <code>students</code> , and <code>enrollments</code> arrays. Run: <code>json-server --watch db.json --port 3000</code>
---------------	---

Task 1: Replace Service Data with HttpClient Calls

Goal: Refactor `CourseService` and `EnrollmentService` to use real HTTP calls.

Steps:

78. Import `HttpClientModule` in your app (or `provideHttpClient()` in `app.config.ts` for standalone Angular). Inject `HttpClient` into `CourseService`: `constructor(private http: HttpClient) {}`.
79. Replace the hardcoded `courses` array with an HTTP call: `getCourses(): Observable<Course[]> { return this.http.get<Course[]>('http://localhost:3000/courses'); }`. Do the same for `getCourseById`, returning `Observable<Course>`.
80. In `CourseListComponent`, update the `ngOnInit` to subscribe: `this.courseService.getCourses().subscribe({ next: courses => this.courses = courses, error: err => this.errorMessage = err.message, complete: () => this.isLoading = false });`.
81. Add a POST method to `CourseService`: `createCourse(course: Omit<Course, 'id'>): Observable<Course> { return this.http.post<Course>('http://localhost:3000/courses', course); }`. Wire it to the enrollment form's submit handler.
82. Add PUT and DELETE methods: `updateCourse` and `deleteCourse`. Test all four operations using Postman or the Angular app UI.

Hint:

- `HttpClient` methods return cold Observables — they do not execute until subscribed. Always unsubscribe or use the `async` pipe to avoid memory leaks.
- Use the `async` pipe in templates (`{{ courses$ | async }}`) to automatically subscribe and unsubscribe — it is the cleanest pattern for displaying Observable data.

Expected Outcome: Course list loads from JSON Server. Adding a course via POST persists it to `db.json`. DELETE removes it. All operations reflected in the UI.

Task 2: RxJS Operators and Error Handling

Goal: Apply RxJS operators to transform and handle HTTP responses robustly.

Steps:

83. Chain the `map` operator to transform the API response before it reaches the component: `getCourses().pipe(map(courses => courses.filter(c => c.credits > 0)))`. Import `map` from `'rxjs/operators'`.
84. Add error handling using `catchError`: `getCourses().pipe(catchError(err => { console.error(err); return throwError(() => new Error('Failed to load courses. Please try again.')). })`. In the component, display the error message in the template.

85. Add a tap operator for side effects (logging): `getCourses().pipe(tap(courses => console.log('Courses loaded:', courses.length)), catchError(...))`. Explain in a comment why tap is preferred over side effects inside map.
86. Implement a retry strategy: `getCourses().pipe(retry(2), catchError(...))`. This retries failed HTTP requests up to 2 times before propagating the error.
87. Use `switchMap` to chain two HTTP calls: when a course is selected, load its enrolled students — `switchMap(courseId => this.enrollmentService.getStudentsByCourse(courseId))`. Explain in a comment why `switchMap` cancels the previous inner Observable when a new `courseId` arrives.

Hint:

- `switchMap` is essential for type-ahead search and dependent HTTP calls — it cancels the previous request if a new one arrives before the first completes, preventing out-of-order responses.
- `tap` is for side effects (logging, analytics) that should not alter the stream. Never modify data inside `tap` — use `map` for transformations.

Expected Outcome: Courses load with error handling — turning off JSON Server shows the error message. retry attempts 2 retries before showing the error. `switchMap` cancels previous course student requests.

| Task 3: HTTP Interceptors

Goal: Build interceptors to handle auth tokens and global error logging.

Steps:

88. Generate an interceptor: `ng generate interceptor interceptors/auth`. In the `intercept` method, clone the request and add an Authorization header: `req.clone({ setHeaders: { Authorization: 'Bearer mock-token-12345' } })`. Register it in `app.config.ts`: `provideHttpClient(withInterceptors([authInterceptor]))`.
89. Verify the interceptor works: open Chrome DevTools → Network → select any API call → check the Request Headers for the Authorization header.
90. Generate a second interceptor: `ng generate interceptor interceptors/error-handler`. In it, use `catchError` to intercept HTTP errors globally: if status 401, navigate to login; if status 500, show a global error notification. Return `throwError` to propagate after handling.
91. Generate a loading interceptor: `ng generate interceptor interceptors/loading`. Use a `LoadingService` (which you also create) with a `isLoading$ BehaviorSubject`. Set it to true in `intercept` before `next.handle(req)`, and false in the pipe's `finalize` operator. Bind a global loading spinner to `loadingService.isLoading$` using the `async` pipe.

Hint:

- Interceptors run in the order they are registered. The request goes through interceptors in registration order; the response travels back in reverse order.
- `finalize` in RxJS runs whether the Observable completes or errors — it is the correct place to hide a loading spinner, equivalent to a `try/catch/finally` block.

Expected Outcome: DevTools shows Authorization header on all API requests. Simulating a 401 response navigates to home. A spinner appears on every HTTP call and disappears when it completes.

HANDS-ON 9 [Advanced]**State Management — NgRx Store, Actions, Reducers, Effects & Selectors****Topics Covered:**

✓ When to Use NgRx vs Services	✓ Actions & Action Creators
✓ Reducers & Immutable State	✓ Selectors & Memoisation
✓ NgRx Effects for Async Operations	✓ RxJS Observables & Operators in NgRx Context

As Angular applications grow, managing shared state in services becomes hard to maintain and debug. NgRx brings Redux-pattern state management to Angular with powerful developer tooling. You will migrate the course and enrollment state to NgRx.

INSTAL	<code>npm install @ngrx/store @ngrx/effects @ngrx/entity @ngrx/store-devtools</code> — also install Redux DevTools Chrome extension
---------------	---

Task 1: Set Up NgRx Store and Define Course State

Goal: Configure the NgRx store and create actions, reducer, and selectors for courses.

Steps:

92. Import `StoreModule.forRoot({})` in `AppModule` (or `provideStore()` in `app.config.ts`). Import `StoreDevtoolsModule.instrument({ maxAge: 25 })` for debugging.
93. Create the course state files inside `store/course/`: `course.actions.ts`, `course.reducer.ts`, `course.selectors.ts`. In `course.actions.ts`, define: `loadCourses = createAction('[Course] Load Courses')`, `loadCoursesSuccess = createAction('[Course] Load Courses Success', props<{ courses: Course[] }>())`, `loadCoursesFailure = createAction('[Course] Load Courses Failure', props<{ error: string }>())`.
94. In `course.reducer.ts`, define the state interface: `{ courses: Course[], loading: boolean, error: string | null }`. Create the reducer using `createReducer` with `on()` handlers for all three actions.
95. Register the reducer: `StoreModule.forFeature('course', courseReducer)` (or `provideState` in standalone). In `course.selectors.ts`, create: `selectCourseState`, `selectAllCourses`, `selectCoursesLoading`, `selectCoursesError` using `createFeatureSelector` and `createSelector`.
96. In `CourseListComponent`, replace the service subscription with: `this.courses$ = this.store.select(selectAllCourses)`. Dispatch the load action in `ngOnInit`: `this.store.dispatch(loadCourses())`. Render using the async pipe: `<app-course-card *ngFor='let c of courses$ | async'>`.

Hint:

- The `[Course]` prefix in action type strings is a convention that groups actions by feature — it makes the Redux DevTools timeline readable: you can filter by `'[Course]'` to see only course-related actions.
- Selectors are memoised — they only recompute when their input selectors change. This is NgRx's key performance optimisation.

Expected Outcome: Redux DevTools shows the `loadCourses` action dispatched on page load. State tree shows `courses`, `loading`, and `error`. Courses render via the async pipe.

Task 2: NgRx Effects for HTTP and Enrollment State

Goal: Use NgRx Effects to handle async API calls and add enrollment state to the store.

Steps:

97. Create `course.effects.ts`. Define a `loadCourses$` effect: `createEffect(() => this.actions$.pipe(ofType(loadCourses), switchMap(() => this.courseService.getCourses().pipe(map(courses => loadCoursesSuccess({ courses }))),`

`catchError(error => of(loadCoursesFailure({ error: error.message })))))))). Register CourseEffects in EffectsModule.forRoot([CourseEffects]) or provideEffects.`

98. Verify the full flow: dispatching loadCourses → Effect fires HTTP → loadCoursesSuccess dispatched → reducer updates state → selector emits new value → component re-renders. Trace each step in Redux DevTools.
99. Add enrollment state: create store/enrollment/ with actions (enrollInCourse, unenrollFromCourse, setEnrolledCourses), a reducer managing enrolledCourseIds: number[], and selectors (selectEnrolledIds, selectEnrolledCourses using a cross-slice selector combining course and enrollment state).
100. Dispatch enrollInCourse({ courseId: id }) from CourseCardComponent when Enroll is clicked. Use selectEnrolledIds to show/hide the Unenroll button: `{{ (enrolledIds$ | async)?.includes(course.id) ? 'Unenroll' : 'Enroll' }}`.

Hint:

- Effects are the only place in NgRx where side effects (HTTP calls, navigation, localStorage) should happen — reducers must remain pure functions.
- Cross-slice selectors (combining course and enrollment state) are a powerful NgRx pattern — use createSelector with multiple input selectors to derive joined data without duplicating state.

Expected Outcome: Redux DevTools shows the complete action → effect → success flow. Enrollment state updates correctly. The async pipe renders enrolled/unenrolled state per card.

HANDS-ON 10 [Advanced]**Unit Testing Angular Applications — Jasmine, Karma & TestBed****Topics Covered:**

✓ Jasmine Syntax — describe, it, expect, spyOn	✓ TestBed for Angular Testing
✓ Component Testing — fixture, debugElement	✓ Testing @Input and @Output
✓ Service Testing with HttpClientTestingModule	✓ Component + Store Testing with MockStore

Every spec.ts file Angular generated has been waiting — now you will actually write tests. This hands-on covers unit testing Angular components and services using Jasmine, Karma, and Angular's TestBed utilities.

RUN	ng test — runs all spec.ts files in watch mode using Karma. Press Ctrl+C to stop. Run ng test --code-coverage to generate a coverage report in coverage/ folder.
------------	--

Task 1: Testing a Component — CourseCardComponent

Goal: Write unit tests for CourseCardComponent covering rendering, inputs, and output events.

Steps:

101. Open course-card.component.spec.ts. Inside the existing describe block, add a beforeEach that configures TestBed: `TestBed.configureTestingModule({ declarations: [CourseCardComponent] })` (or imports for standalone). Call `fixture = TestBed.createComponent(CourseCardComponent); component = fixture.componentInstance;`
102. Write a test that verifies the component is created: `it('should create', () => { expect(component).toBeTruthy(); });`
103. Write a test for @Input rendering: set `component.course = { id:1, name:'Data Structures', code:'CS101', credits:4, gradeStatus:'passed' }`; `fixture.detectChanges();` then use `fixture.debugElement.query(By.css('h3')).nativeElement.textContent` to assert the course name is displayed.
104. Write a test for @Output: set `component.course = mockCourse;` `fixture.detectChanges();` `spyOn(component.enrollRequested, 'emit');` `fixture.debugElement.query(By.css('button')).nativeElement.click();` `fixture.detectChanges();` `expect(component.enrollRequested.emit).toHaveBeenCalledWith(1);`
105. Write a test for ngOnChanges: call `component.ngOnChanges` with a SimpleChanges object and verify the expected console.log was called (spy on console.log).

Hint:

- `fixture.detectChanges()` triggers Angular's change detection — always call it after changing component properties before querying the DOM.
- `By.css()` (from `@angular/platform-browser`) is the Angular way to query the rendered DOM in tests — use it instead of `document.querySelector` to stay within the component's scope.

Expected Outcome: ng test shows 5 passing tests for CourseCardComponent. Coverage report shows the component's TypeScript logic is covered.

Task 2: Testing a Service and an NgRx-Connected Component

Goal: Test CourseService with HttpClientTestingModule and test an NgRx store-connected component.

Steps:

106. Open course.service.spec.ts. Configure TestBed with HttpClientTestingModule: `TestBed.configureTestingModule({ imports: [HttpClientTestingModule], providers: [CourseService] })`. Inject both the service and HttpTestingController.

107. Write a test for `getCourses()`: call `service.getCourses().subscribe(courses => expect(courses.length).toBe(2))`; then use `httpMock.expectOne('http://localhost:3000/courses').flush(mockCourses)`; and `httpMock.verify()` to assert no unexpected requests were made.
108. Write a test for error handling: flush a 500 error response and assert the Observable emits an error with the expected message.
109. Test a component that uses the NgRx store: configure TestBed with `provideMockStore({ initialState: { course: { courses: mockCourses, loading: false, error: null } } })`. Inject `MockStore`. Call `fixture.detectChanges()` and assert the rendered course cards match the initial state.
110. Use `store.setState({ course: { courses: [], loading: true, error: null } })` to simulate a loading state. Call `fixture.detectChanges()` and assert the loading indicator is visible in the DOM.

Hint:

- `HttpTestingController.verify()` asserts that there are no outstanding (unsatisfied) HTTP requests after each test — critical for catching tests that make unintended HTTP calls.
- `MockStore` from `@ngrx/store/testing` replaces the real store with a controllable mock — you can set the state and dispatch actions without running real reducers or effects.

Expected Outcome: ng test shows all service tests passing. The `getCourses` test verifies the correct URL was called. The `MockStore` component test shows loading indicator when loading is true.